Professional Elective-II

COMPILER DESIGN (BCSPE601)

Course Objective:

The objective of the course is to introduce the major concept areas of language translation and compiler design, enrich the knowledge in various phases of compiler ant its use, code optimization techniques, machine code generation, and use of symbol table, extend the knowledge of parser by parsing LL parser and LR parser, and provide practical programming skills necessary for constructing a compiler.

Module I

Compiler Overview and Lexical Analysis: Overview of language processing: preprocessors, compiler, assembler, interpreters, linkers, Bootstrap loaders and cross compiler. Structure of a compiler: phases of a compiler. Lexical Analysis: Role of Lexical Analysis, Input buffering, Regular Expressions, NFA, DFA, Minimization of DFA, Transition diagram for tokens, reserved words and identifiers. Lexical error and its recovery, LEX.

Module II

Syntax Analysis Role of a parser, Top down parsing, derivation, ambiguity, left recursion, left factoring, backtracking parsing, recursive decent parsing, predictive parsing, LL(1) Grammars. Bottom up parsing, Shift Reduce Parsing, handle, handle pruning, Operator precedence parser, precedence function, LR Parsers, Construction of SLR, CLR, LALR Parsing tables, parser conflicts, Dangling ELSE Ambiguity, Error recovery in Parsing. YACC.

Module III

Semantic Analysis and Intermediate code generation: Semantic analysis, SDD and SDTS, evaluation of semantic rules, implementation of S-attributed and L-attributed definition. Type analysis and type checking. Intermediate code, three address code, quadraples, triples, indirect triplet, abstract syntax trees, Declarations, Assignment statements, Boolean expressions, Case statements, Back patching, Procedure calls. Symbol tables, use of symbol tables. Runtime Environment: storage organization, stack allocation, access to non-local data, heap management, parameter passing mechanisms.

Module IV

Optimization: Machine independent code optimization: Common sub expression elimination, constant folding, copy propagation, dead code elimination, strength reduction, loop optimization, basic blocks, data flow analysis.

Module V

Code generation: Issues in the design of code generation, The target machine, A simple code generator, DAG representation of basic blocks. Machine dependent code optimization: Peephole optimization, register allocation, instruction scheduling, inter procedural optimization, garbage collection via reference counting.

Text books:

1. Compilers, Principles Techniques and Tools- Alfred V Aho, Monical S Lam, Ravi Sethi, Jeffrey D. Ullman,2 nded, Pearson,2007.
2. Principles of compiler design, V. Raghavan, 2 nded, TMH, 2011.

Reference books:

1. Compiler construction, Principles and Practice, Kenneth C Louden, CENGAGE
2. Compiler Design, O. G. Kakde, University Science Press.
3. Compiler Design, K. Muneeswaram, Oxford University Press.
4. Allen I. Holub "Compiler Design in C", Prentice Hall of India, 2003

Course Outcome:

CO1: Identify the role of compiler and its phases and Role of the lexical analysis
CO2: Role of the top-down and bottom-up parser
CO3: Distinguish s-attributed and l-attributed definition
CO4: Analyze different ways of code optimization
CO5: Issues of a code generator

Course Articulation Matrix

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12
CO1	3	2	2	-	1	-	-	-	3	2	-	-
CO2	3	3	3	1	1	-	1	-	2	2	-	1
CO3	3	2	3	2	2	-	2	2	3	1	-	-
CO4	3	2	1	1	2	-	3	2	3	1	-	-
CO5	3	2	2	2	1	-	1	-	2	1	2	2

1: Slight (Low) 2: Moderate (Medium) 3: Substantial (High) -: No Correlation

Program Articulation Matrix row for this Course

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12
CO	3	2.2	2.2	1.2	1.4	-	1.4	0.8	2.6	1.4	0.4	0.6